Graphics

Introduction

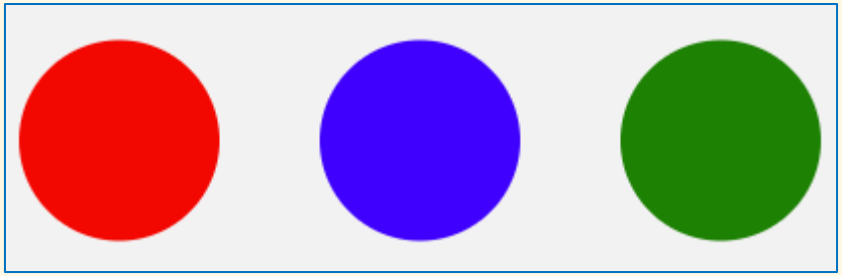
- Graphics are necessary in many applications
- Static graphic contents can be easily achieved using images
- But dynamic contents need to be handled differently:
  - Games
  - Charts that present some data
  - Interactive controls (e.g., a color picker)
  - Anything that moves (e.g., an analog clock)
  - …

React Native Skia Library

- Installation
  - `npm install @shopify/react-native-skia@1.5.0`
    - Currently, versions higher than 1.5.0 are not compatible with Expo
  - On Android:
    - Make sure Android NDK is installed:
      - Android Studio → SDK Manager → SDK Tools → NDK
      - Add an env variable `ANDROID_NDK` that points to the NDK folder
  - On web:
    - `npx setup-skia-web`
    - Do this every time you update the `react-native-skia` package

First Drawing

```
import React from 'react';
import { Canvas, Circle, Group } from '@shopify/react-native-skia';
import { Dimensions } from 'react-native';

export default function SkiaGraphics() {
  const dim = Dimensions.get('window');
  const x0 = dim.width / 2,
        y0 = dim.height / 2;

  return <Canvas style={{flex: 1}}>
    <Circle cx={x0 - 150} cy={y0} r={50} color="red" />
    <Circle cx={x0} cy={y0} r={50} color="blue" />
    <Circle cx={x0 + 150} cy={y0} r={50} color="green" />
  </Canvas>
}
```



Web Support

- Skia needs to be loaded on the browser before importing the Skia module:
  - `import { WithSkiaWeb } from '@shopify/react-native-skia/lib/module/web';`
  -
```
return <WithSkiaWeb
  getComponent={() => import('SkiaGraphics')}
  fallback={<Text>Skia is loading...</Text>}
/>
```

Wrapper Component

- Use a wrapper component to make Skia work seamlessly on every platforms

- SkiaGraphicsWrapper.tsx
  - ```
    import SkiaGraphics from './SkiaGraphics';

    export function SkiaGraphicsWrapper() {
      return <SkiaGraphics />
    }
    ```

- SkiaGraphicsWrapper.web.tsx
  - ```
    import { Text } from 'react-native';
    import { WithSkiaWeb } from '@shopify/react-native-
    skia/lib/module/web';

    export function SkiaGraphicsWrapper() {
      return <WithSkiaWeb
        getComponent={() => import('@/components/SkiaGraphics')}
        fallback={<Text>Skia is loading...</Text>}
      />
    }
    ```

Shapes

- <Points>
- <Line>
- <Path>
- <Circle>
- <Oval>
- <Rect>
- <RoundedRect>
- <DiffRect>
- <Text>

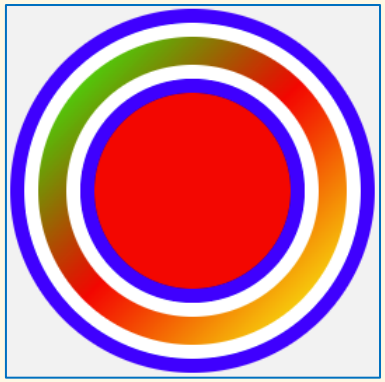
Example


```
const fontFile = require('../assets/fonts/Tangerine-Bold.ttf');
const font1 = useFont(fontFile, 50);
const font2 = useFont(fontFile, 30);

const path = Skia.Path.Make();
path.addCircle(170, 190, 19);

return <Canvas style={{flex: 1}}>
  <Rect x={10} y={10} width={100} height={40} color="red" />
  <RoundedRect x={10} y={60} width={100} height={40} r={10} color="green" />
  <DiffRect outer={rrect(rect(10, 110, 100, 40), 10, 20)}
    inner={rrect(rect(40, 115, 60, 20), 10, 5)} color="blue" />
  <Line p1={vec(10, 200)} p2={vec(110, 160)}
    style="stroke" strokeWidth={4} color="purple" />
  <Points points={[vec(120, 10), vec(220, 50), vec(220, 10), vec(120, 50)]}
    mode="polygon" style="stroke" strokeWidth={4} color="black" />
  <Oval x={120} y={60} width={100} height={40}
    style="stroke" strokeWidth={2} color="magenta" />
  <Text x={120} y={150} text="Hello" font={font1} />
  <TextPath path={path} text="Hello World" font={font2}  />
</Canvas>
```

Painting

- Using properties:
  - `color`, `blendMode`, `style`, `strokeWidth`, `strokeJoin`, `strokeCap`, `strokeMiter`, `opacity`, `antiAlias`
- Using nested painting components:
  - `<Paint>`
  - `<LinearGradient>`, `<SweepGradient>`, `<RadialGradient>`, `<TwoPointConicalGradient>`
  - `<Mask>`, `<BlurMask>`, `<ColorMatrix>`, `<BlendColor>`, `<Lerp>`, `<LinearToSRGBGamma>`, `<SRGBToLinearGamma>`
- Example:

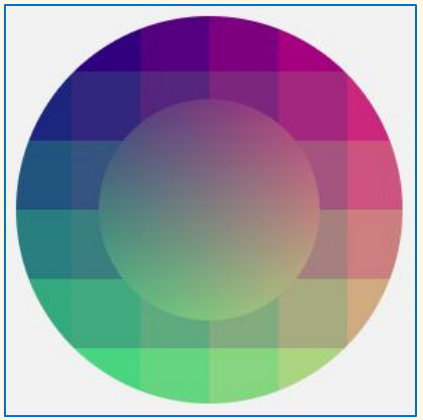
```
<Circle cx={150} cy={150} r={100} style="fill" color="red">
  <Paint style="stroke" strokeWidth={60} color="blue" />
  <Paint style="stroke" strokeWidth={40} color="#fff" />

  <Paint style="stroke" strokeWidth={20}>
    <LinearGradient
      start={vec(50, 50)} end={vec(250, 250)}
      colors={['#1f0', 'red', 'yellow']} />
  </Paint>
</Circle>
```



Shaders

- Skia provides a shading language with a syntax similar to GLSL
- Example:

```
const source1 = Skia.RuntimeEffect.Make(`
  vec4 main(vec2 pos) {
    vec2 norm = pos / vec2(300);
    return vec4(norm.x, norm.y, 0.5, 1);
  }`)!;
const source2 = Skia.RuntimeEffect.Make(`
  vec4 main(vec2 pos) {
    vec2 norm = vec2(
      floor(pos.x / 50) * 50,
      floor(pos.y / 50) * 50
    ) / vec2(300);
    return vec4(norm.x, norm.y, 0.5, 1);
  }`)!;

<Circle cx={150} cy={150} r={110}>
  <Shader source={source1} />

  <Paint style="stroke" strokeWidth={60}>
    <Shader source={source2} />
  </Paint>
</Circle>
```

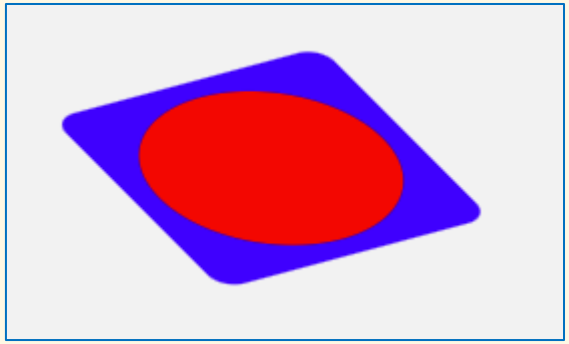


Group

- <Group> can be used to groups multiple elements that share the same attributes, such as transformation, painting, clipping

- Example:

```
<Group color="blue"
  origin={{ x: 100, y: 50 }}
  transform={[
    { rotateZ: -Math.PI / 4},
    { skewX: Math.PI / 6 },
  ]}
>
  <RoundedRect x={40} y={40}
    width={120} height={120} r={10} />
  <Circle cx={100} cy={100} r={50}
    color="red" />
</Group>
```



Animations

- Animations can be achieved in a few ways:
  - Manipulating state variables
  - Using Reanimated package

Exercises

1. Draw a line chart.
2. Draw a bar chart.
3. Create an analog clock as in the following figure.